

FULL-STACK TAKE-HOME

Data-Driven AI Screening Platform

A design-led build for Staff / Lead Full-Stack Engineers

This is not a spec to implement line-by-line. It is a deliberately under-specified problem. We care far more about how you frame it, the decisions you make where we stay silent, and how honestly you account for what you left undone than about how many features you ship. A smaller system with sharp reasoning and a clear-eyed account of its own limits will beat a larger one delivered without rationale.

Expect to spend a focused weekend on this — see §10 *Scope & guardrails*. Depth in a few well-chosen places beats thin coverage everywhere.

Contents

1. The problem, and why it matters

You are building the screening platform for a public-health program. Field health workers run AI-assisted screenings — anemia, malnutrition, respiratory risk, and more — from a web app, often in a clinic tent or a doorway, on a low-end Android phone, over a connection that drops mid-form.

The program revises its screenings constantly. Clinical staff — not engineers — add and change screenings most weeks in response to new guidelines, new cohorts, and field feedback. Waiting on an app release for every change is not an option. So screenings are data-driven: each one is a workflow the backend serves as XML and the frontend renders generically. Adding a screening should require new configuration, not new code.

The stakes are asymmetric and human. A missed anemia case is a child who goes untreated. An over-eager referral floods an already-stretched clinic and erodes trust in the program. The people using this are not the people who built it, they may not share your first language, and they cannot call support when something breaks. Equity, resilience on bad hardware and worse networks, and safe failure are not nice-to-haves here — they are the point.

The slice you own

A vertical full-stack slice of this platform:

- **A backend** that serves workflows by id, runs a mock AI screening step, and persists submitted outcomes.
- **A generic frontend renderer** that consumes any conforming workflow and drives the whole encounter from it.

The AI step is simulated — no real model. But design as if a real model will replace the mock next quarter with no frontend change.

2. What we are actually evaluating

We are transparent about this because we want your best thinking, not a guessing game about our preferences. This exercise is built to surface five things:

We look for	What strong looks like
Problem framing & scoping	You decide what to build, what to defer, and what to fake — and you can say why each was the right cut for this problem.
Design & trade-offs	Consequential choices are explicit, alternatives are named, and you can defend the call you made against the one you rejected.
Judgment under ambiguity	Where the brief is silent (and it is silent on purpose), you make a reasoned call and own it rather than waiting to be told.
Awareness of completeness	You know your own gaps. You can name what would break first, your riskiest assumption, and what you knowingly left out.
Engineering rigor	The hard parts are actually correct — idempotency, validation, safe rendering — and your testing effort is aimed where it matters.

What we are *not* grading: pixel-perfect UI, breadth of features, your choice of framework, or lines of code. Polish spent on the wrong thing is itself a signal.

3. What to build — the core slice

Four deliverables form the runnable core. Everything after §3 is about how well you reason around this core, not about adding more surface to it.

3.1 A workflow format (you design it)

An XML format, of your own design, that fully describes an AI-assisted screening: an ordered set of steps, the inputs collected, the AI screening step (everything needed to invoke the model and interpret its result), branching, and terminal outcomes. Document the format so a clinical author could — in principle — read your docs and write a new screening.

3.2 A backend service (Python and/or Node)

- Serve a workflow by id.
- Expose a mock AI screening endpoint that accepts the step's inputs and returns a simulated result — score / label / confidence.
- Persist a submitted screening outcome.
- Document the API contract (an OpenAPI sketch or a clear README section).

3.3 A generic frontend renderer (any web stack)

The renderer fetches a workflow, renders each step's UI from the definition, collects and validates inputs, calls the AI screening endpoint at the AI step, branches on prior answers and the AI result, and submits the final outcome. It must be genuinely generic — it interprets the workflow; it is not hard-coded to any one screening.

3.4 Two different sample workflows on one renderer

Two screenings that are structurally different — for example an image/value-based anemia screening and a questionnaire-based risk screening — both running unmodified on your single renderer. This is how you prove the platform is genuinely config-driven and not a demo with a coat of abstraction.

4. Deliberate ambiguities and tensions

Read this section as the heart of the exercise. We have left the questions below unanswered on purpose. Most of them pull against each other — resolving one well tends to cost you on another. There is no answer key. We want to see the call you make, the cost you accept, and the reasoning that got you there. Engaging three of these deeply beats gesturing at all seven.

a. Expressiveness vs. safety

The richer your workflow format, the more power clinical authors have — and the larger the surface for injection when untrusted XML is rendered into the DOM. Where do you draw the line between a format that can express real screenings and one you can actually keep safe?

b. Authorability vs. rigor

Clinical staff, not engineers, author these workflows. How much structure, schema, and validation can you impose before the format becomes unauthorable by a non-engineer — and how little before it becomes unsafe or ambiguous?

c. Forward & backward compatibility

An old frontend in the field meets a workflow authored for a newer engine — and vice versa. When the renderer meets an element it does not understand, does it fail closed (refuse) or degrade (skip and continue)? Who is allowed to decide that per element, and how is it expressed?

d. Client convenience vs. trust

Branching on prior answers and the AI result is easiest on the client — but the client is untrusted, offline, and possibly tampered with. Can a field worker's device be trusted to compute and submit an outcome? Where does authority actually live, and what does that cost you in offline capability?

e. Idempotency vs. real-world retries

Field clients on bad networks retry. But a retry may carry a mutated payload, a duplicate key, or a skewed clock; and a genuine re-screen of the same patient looks a lot like a duplicate. Define precisely what 'the same submission' means, what your idempotency key covers, and where its guarantee ends.

f. Generic renderer vs. humane UX

A fully generic renderer trends toward a lowest-common-denominator UI. But your users have low literacy, low-end devices, and no patience for a confusing form. How do you keep the renderer screening-agnostic while still producing something a stressed health worker can actually use?

g. Offline-first vs. freshness

You will want to cache workflows so they run offline. But a workflow can be revised — or urgently retired for a safety fix — between the moment a client caches it and the moment a worker submits against it. How fresh is fresh enough, and who wins?

5. Scenarios your design must survive

These are concrete situations the platform will actually meet. You do not have to build for all of them — but your design should have an answer, in code or in writing, and you should be honest about which you handled and which you knowingly deferred.

Scenario	The question it forces
Chained AI steps	A screening needs two AI steps, where the second consumes the structured output of the first (e.g., a triage classifier feeds a severity estimator). Does your format and renderer handle multi-AI flows — or did an assumption of 'one AI step' quietly bake itself into the design?
Version drift mid-encounter	A worker opens screening v3. Before they submit, v4 is published and v3 is retired for a safety correction. What happens to the in-flight encounter — does it complete on v3, block, silently upgrade, or something else? Who is accountable for the outcome?
Low confidence & contradiction	The AI returns low confidence — or a result that contradicts a prior answer (the questionnaire says severe; the AI says normal). What is the safe fallback, how does the workflow express it, and who makes the final call?
Population calibration / equity	A confidence threshold tuned on one population misfires on another. Can thresholds vary by locale or cohort? Does that live in the XML, in the model, or somewhere else — and what does that choice imply for authoring and safety review?
Duplicate vs. genuine re-screen	Two outcomes arrive for the same patient encounter: one is a network retry, one is a real second screening minutes later. How does the backend tell them apart without dropping the real one or storing the duplicate?
Localization vs. canonical values	Step labels come from the workflow and may be localized, including right-to-left scripts. But the AI expects canonical input values, not the localized display text the worker sees. How do you keep display and semantics separated end-to-end?
Then go further. List at least three failure modes we did not mention. For each, say whether you handled it, mitigated it, or knowingly deferred it — and why. The failure modes you surface unprompted tell us more than the ones we hand you.	

6. Hard requirements (non-negotiable)

Everything above is about judgment. These are the invariants — the platform is not credible without them.

- 1. The workflow drives the AI step entirely.** At minimum the XML carries: which model/screening is invoked and a version constraint; which collected inputs feed it; how its outputs bind back into the workflow; any parameters/thresholds; and what to do on low confidence or error. Adding a new screening requires new XML only, never new code.
- 2. The renderer is generic.** Switching workflows — or editing one — changes the flow with no screening-specific code change.
- 3. The AI call goes through one clearly-defined backend endpoint (mocked)** so a real model can replace the mock with no frontend change.
- 4. Result submission is idempotent.** Field clients on bad networks will retry; a retried submission must not create duplicates. Define your idempotency key and the boundary of its guarantee.
- 5. Unhappy paths are handled:** input validation, AI low-confidence, AI error, and a safe fallback (e.g., retake or manual referral).
- 6. Branching is driven by both prior answers and the AI result** — not one or the other.
- 7. The workflow is untrusted data.** Prevent script injection / XSS when rendering it into the DOM, and validate workflows server-side before serving them.
- 8. It runs locally with one documented command.** Docker Compose welcome. No cloud, no paid services. (No automated tests required — but read §8 on testing judgment.)

7. Think 360° — address in code or in writing

Breadth of consideration, not breadth of implementation. Most of these should be reasoned about in your writing; implement only what earns its place in the core.

Dimension	What to address
Extensibility	How a new step type or input type is added without breaking existing workflows or forcing a coordinated frontend release.
Format & API versioning	How an older frontend handles a newer workflow (and vice versa); how the API evolves without breaking clients already in the field. Unknown elements: fail closed or degrade — decide, per element or globally, and justify.
Security & threat model	A workflow is untrusted data rendered into the DOM. Beyond XSS: what can a malicious or careless author do, what can a tampered client do, and how do you validate a workflow server-side before it ever ships?
Scale, offline & equity	Fanning out to many clients on poor networks: caching workflows, bursty/duplicate submissions, low-end devices, bundle size, internationalization, accessibility. Who is excluded by each choice you make?
Observability & operability	A screening fails for a worker you cannot reach, on a device you cannot see, with no repro. How would on-call diagnose it? What would you need to have logged?
Testing strategy	What you would test, what you would deliberately not test, and why. At staff level, knowing where testing is not worth it is as much a signal as coverage.

Dimension	What to address
Authoring & provisioning (describe, don't build)	How non-engineers author workflows safely; how a workflow and its referenced model version are delivered and updated to field clients; and how you would roll back a bad screening fast.

8. Required written artifacts

This is where staff-level candidates separate themselves. Keep these tight — bullet-level reasoning is welcome. We value honesty and precision over volume, and we would rather read one page of real thinking than five of restated requirements.

8.1 README

Your workflow format documented, the API contract, run instructions, the design rationale, how the renderer stays generic, and your §7 notes.

8.2 Decision log

The five to eight most consequential decisions you made. For each: the decision, the alternatives you seriously considered, and why you rejected them. If a decision was forced by time rather than conviction, say so.

8.3 Trade-off register

For each tension in §4 you engaged with: the call you made and what it costs you. It is entirely acceptable to write 'I chose X and knowingly gave up Y' — that is the point.

8.4 Threat model note

What a malicious workflow author or a tampered client could attempt, ranked by plausibility and blast radius, and your mitigations (or your reason for accepting the risk).

8.5 Limits & next

Explicit and unsparing:

- What you left out, and why it was the right thing to cut.
- What breaks first at 10× and at 100× the load or workflow count.
- The single riskiest assumption you made.
- What you would build next given another week.

9. Scope and guardrails

Over-engineering is a signal too. We are not looking for someone who builds everything in §4 and §5. We are looking for someone who builds a credible, correct core, chooses a few hard problems to go deep on, and writes clearly about the rest. Tell us what you deliberately kept simple and why.

- **Time-box:** a focused weekend. If you are spending more, you are probably building where you should be writing.
- **Depth over breadth:** thin coverage of every dimension reads as weaker than deep, honest treatment of a few.
- **Fake convincingly:** the AI is mocked, and other things can be too — but make your seams explicit so we can see where a real system would slot in.

- **No automated tests are required**, but your §8.5 and §7 testing notes should show you know exactly what you would test if this were going to production.

10. How we will evaluate

We publish the rubric on purpose. We want your strongest reasoning aimed at what matters, not spent decoding what we want.

Dimension	Weak	Strong
Problem framing & scoping	Builds to the letter of the brief; no cuts, no priorities.	Reframes the problem, cuts deliberately, and defends the cuts.
Design & trade-offs	Choices are implicit or unexamined; alternatives unmentioned.	Choices are explicit, alternatives named, costs owned.
Ambiguity handling	Waits to be told; picks the first workable path.	Makes reasoned calls where the brief is silent and stands behind them.
Correctness on hard parts	Idempotency, validation, or XSS handled superficially or wrongly.	The invariants in §6 are genuinely, demonstrably correct.
Generic renderer	Abstraction leaks; a second workflow would need code.	Two structurally different workflows run unmodified; the seam is clean.
Communication & gap-awareness	Docs restate requirements; gaps unacknowledged.	Writing is sharp; limits, risks, and next steps are named without prompting.

11. What to submit

- **A git repo (link or zip)** that runs locally with one documented command.
- **The written artifacts from §8** — README, decision log, trade-off register, threat model, limits & next.
- **Your two sample workflow XML files** (§3.4).
- **Optional:** a short screen recording of both workflows running.

12. The follow-up conversation

Build as if the next step is a conversation — because it is. In a live session we will ask you to:

1. **Defend a decision to a skeptical reviewer** — we will pick one choice from your decision log and push on it.
2. **Extend the system live** — for example, add a new step type, a new input type, or a new branching rule while we watch. Design for change, and be ready to explain any line you wrote.

There are no trick questions here and no single right answers. We are looking for an engineer we would trust to own an ambiguous, high-stakes system and to tell us the truth about its state. Show us that.